




**NATIONAL ECONOMICS UNIVERSITY**

**NEU COLLEGE OF TECHNOLOGY (NCT)**

**FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE**


# COURSE PROJECT REPORT

## INTRODUCTION TO ARTIFICIAL INTELLIGENCE

---

## TOPIC: SOLVING THE N-QUEENS PROBLEM ($4 \leq N \leq 8$)

*Systematic State-Space Search, Constraint Propagation, and Local Optimization*

---


| | | |
|---|---|---|
| **Instructor:** | Dr. Nguyen Trong Nghia | |
| **Project Group:** | Group 08 | |
| **Team Members:** | 1. Duong Minh Nguyen | [11256308] |
| | 2. Vu Hoang Son | [11257428] |
| | 3. Dang Dinh Gia Bao | [11251193] |
| **Academic Term:** | Semester 1, Academic Year 2026–2027 | |
| **Repository:** | https://github.com/wannabeAIE0277/N-Queens-Problem | |



**HANOI, OCTOBER 2026**

# ABSTRACT

The N-Queens problem is an established benchmark in Artificial Intelligence (AI) for investigating combinatorial search, algorithmic complexity, and heuristic optimization. This report delivers an exhaustive formal analysis and algorithmic implementation of the problem within the experimental range $4 \leq N \leq 8$. We examine the theoretical boundary separating Uninformed Depth-First Search (Blind DFS) from Backtracking Search enhanced with constant-time $O(1)$ constraint pruning. Furthermore, we expand the problem into modern AI methodologies: Constraint Satisfaction Problems (CSP) equipped with Forward Checking and Minimum Remaining Values (MRV), and Local Search driven by the Min-Conflicts heuristic with random restarts. We introduce a modular software architecture alongside vector-rendered search trees illustrating pruning mechanics. Empirical results across all configurations substantiate that early constraint filtering reduces the candidate search space by more than 99.9%, completely enumerating all 92 distinct solutions of the $8 \times 8$ board in mere milliseconds with an optimal $O(N)$ linear memory footprint.

## Project Task Distribution and Contribution

Table 1: Team member roles, specific work packages, and contribution workload distribution

| Member Name | Assigned Work Packages & Responsibilities | Outputs | Contrib. |
|---|---|---|---|
| Duong Minh Nguyen | **Lead across Packages :** Mathematical modeling, DFS Backtracking ($O(1)$), CSP (FC), Min-Conflicts local search, benchmark runner, GUI visualizer, LaTeX report lead, and Git administration. | Full Codebase, Test Suite, LaTeX Report | 50% |
| Vu Hoang Son | **Packages:** Benchmark execution, empirical runtime data collection, performance analysis, presentation structure, and oral defense. | Benchmark Data, Presentation Slides | 30% |
| Dang Dinh Gia Bao | **Packages:** Metric log cross-checking, empirical data verification, presentation layout & visual design, rehearsal, and defense support. | Data Logs, Presentation Deck | 20% |

**Declaration on the Use of Generative AI Tools:**
In accordance with academic integrity guidelines, the authors declare that **Claude** (Anthropic) was utilized as an interactive programming assistant during the development phase. Specifically, Claude assisted with code scaffolding, syntax checking, and generating initial boilerplate scripts for testing and plotting. All core mathematical formulations, algorithmic logic, experimental pipelines, verification tests, and analytical interpretations were conceptualized, directed, verified, and finalized entirely by the human authors, who assume full responsibility for this work.

# Contents

**1 Introduction and Problem Formulation** **3**
1.1 Historical Background and Motivation . . . 3
1.2 Scope and Experimental Boundaries . . . 3
1.3 Practical Engineering Applications . . . 3
1.4 Open-Source Code Repository . . . 3

**2 Mathematical Modeling and State-Space Formulation** **4**
2.1 Formal Search Formulation and State Representation . . . 4
2.2 Constraint Equations and $O(1)$ Verification . . . 4
2.3 Benchmark Statistics, Symmetry, and Complexity Analysis . . . 5

**3 Depth-First Search and Backtracking** **6**
3.1 Uninformed DFS vs. Systematic Backtracking . . . 6
3.2 Why Not Breadth-First Search (BFS) and Its Variants? . . . 6
3.3 Search Tree Pruning Mechanics . . . 7
3.4 Algorithmic Implementation with $O(1)$ Checks . . . 7
3.5 Software System Architecture . . . 8

**4 Extended AI Search Paradigms** **9**
4.1 Constraint Satisfaction Problem (CSP) Formulation . . . 9
4.2 Local Search and Min-Conflicts Heuristic . . . 10
4.3 Informed Search and Evolutionary Approaches . . . 10
4.3.1 A* Heuristic Search . . . 10
4.3.2 Genetic Algorithm (GA) Formulation . . . 11

**5 Experimental Evaluation & Comparative Analysis** **12**
5.1 Experimental Setup and Benchmark Methodology . . . 12
5.2 Empirical Benchmark Results . . . 12
5.3 Performance Visualization . . . 13
5.4 Analytical Discussion & Findings . . . 13
5.5 Qualitative Trade-off Summary & Study Limitations . . . 14

**6 Conclusion & Future Work** **15**
6.1 Summary of Contributions . . . 15
6.2 Key Lessons Learned . . . 15
6.3 Future Research Directions . . . 15

# 1. Introduction and Problem Formulation

## 1.1 Historical Background and Motivation

The eight-queens puzzle was originally devised in 1848 by chess composer Max Bezzel [2]. Soon after, eminent mathematician Carl Friedrich Gauss engaged in determining its complete set of fundamental solutions. In modern computer science, this puzzle was generalized to an arbitrary chessboard of dimension $N \times N$, universally termed the **N-Queens Problem** [1]. Formally, the objective is to place $N$ chess queens on an $N \times N$ grid such that no two queens threaten each other along any shared row, column, or diagonal. It represents a canonical model for investigating combinatorial explosions, state-space formulations, and constraint satisfaction algorithms [3, 4].

## 1.2 Scope and Experimental Boundaries

Our investigation rigorously targets the board dimension scale $4 \leq N \leq 8$:

- **Boundary $N < 4$:** Trivially solved ($N = 1$) or mathematically impossible ($N = 2, 3$) due to immediate geometric confinement.
- **Target Range $4 \leq N \leq 8$:** Compact enough to allow exhaustive enumeration of all valid goal states and trace complete execution trees, yet sufficiently complex to exhibit exponential state growth and substantiate the necessity of intelligent pruning [2].

## 1.3 Practical Engineering Applications

Far from an isolated recreational puzzle, the underlying constraint-satisfaction and conflict-resolution mechanics directly apply to real-world engineering challenges [2, 6]:

- **VLSI Chip Floorplanning:** Arranging functional macro-blocks on integrated circuits to prevent signal cross-talk and routing congestion.
- **Parallel Task & Memory Scheduling:** Allocating concurrent processes across multi-core processors and memory banks without bus contention.
- **Wireless & Optical Channel Assignment:** Assigning frequencies or optical routing paths across mesh networks without mutual interference.

## 1.4 Open-Source Code Repository

To guarantee full scientific reproducibility, our complete software package, automated unit tests, benchmark runners, and raw CSV logs are archived on GitHub:

https://github.com/wannabeAIE0277/N-Queens-Problem-Introduction-to-Artificial-Intelligent-.git

# 2. Mathematical Modeling and State-Space Formulation

## 2.1 Formal Search Formulation and State Representation

Following the classical state-space search framework established by Russell and Norvig [1], the N-Queens problem is formally defined as a 6-tuple $(S, s_0, A, \text{Result}, \text{IsGoal}, c)$ under our **Incremental (Constructive) Formulation**:

- **State Space ($S$):** Any partial assignment vector $s = [x_1, x_2, \ldots, x_k]$ of length $k$ ($0 \leq k \leq N$), where $x_i \in \{1, \ldots, N\}$ denotes the column position of the queen placed in row $i$.
- **Initial State ($s_0$):** The empty board configuration $s_0 = [\,]$ with $k = 0$ queens placed.
- **Actions ($A(s)$):** Given a state at row depth $k$, the legal actions consist of selecting a safe column $c \in \{1, \ldots, N\}$ for row $k+1$ that does not conflict with any previously placed queen $x_1, \ldots, x_k$.
- **Transition Model (Result$(s, a)$):** Appending the chosen column $c$ to the partial vector, yielding $s' = [x_1, \ldots, x_k, c]$ at depth $k+1$.
- **Goal Test (IsGoal$(s)$):** Returns `True` if and only if $k = N$ and all $N$ queens are mutually non-attacking.
- **Path Cost ($c$):** Uniform step cost of 1 per queen placement, resulting in a fixed path cost of $N$ for every valid goal state.

The choice of state representation dictates the combinatorial size of the candidate space:

1. **Naive 2D Grid Formulation:** Treating the board as $N^2$ unconstrained squares yields $|S_{\text{naive}}| = \binom{N^2}{N}$, producing $4,426,165,368$ states for $N = 8$.
2. **Row-Indexed 1D Vector Formulation:** Enforcing one queen per row reduces the space to $N^N$ ($16,777,216$ states for $N = 8$), and enforcing distinct columns reduces it to $N!$ permutations ($40,320$ states for $N = 8$).

## 2.2 Constraint Equations and $O(1)$ Verification

Let $(i, x_i)$ denote a queen placement at row $i$, evaluated against an antecedent queen $(j, x_j)$ where $1 \leq j < i$. The configuration is conflict-free if and only if:

$$x_i \neq x_j \quad \text{(Column Independence)} \quad \wedge \quad |x_i - x_j| \neq |i - j| \quad \text{(Diagonal Independence)} \tag{1}$$

Iterating over all $j \in \{1, \ldots, i-1\}$ naively requires $O(N)$ operations per check. To achieve $O(1)$ constant-time verification [3], we project diagonal coordinates onto 1D boolean lookup arrays:

- **Main Diagonal (Slope $+1$):** Cells $(r, c)$ on the same descending diagonal share an invariant difference $(r - c)$. Adding an offset $N$ maps $(r - c + N)$ into a direct boolean array index.
- **Anti-Diagonal (Slope $-1$):** Cells on the same ascending diagonal share an invariant coordinate sum $(r + c)$, indexing directly into a second boolean array.

## 2.3 Benchmark Statistics, Symmetry, and Complexity Analysis

While systematic search enumerates all valid permutations (Total Solutions), many configurations are geometrically equivalent under the dihedral group $D_4$ via rotations $(90°, 180°, 270°)$ and reflections [2]. Table 2 and Figure 1 contrast the growth of solution counts alongside state-space expansion.

Table 2: Theoretical states and solution counts $(4 \leq N \leq 8)$

| $N$ | $N^N$ | $N!$ | **Fund.** | **Total** |
|---|---|---|---|---|
| 4 | 256 | 24 | 1 | 2 |
| 5 | 3,125 | 120 | 2 | 10 |
| 6 | 46,656 | 720 | 1 | 4 |
| 7 | 823,543 | 5,040 | 6 | 40 |
| 8 | 16,777,216 | 40,320 | 12 | 92 |

**Note:** Enforcing permutation constraints reduces the candidate space by a factor of $\frac{N^N}{N!}$, scaling from $10.7\times$ $(N = 4)$ to $416.1\times$ $(N = 8)$.

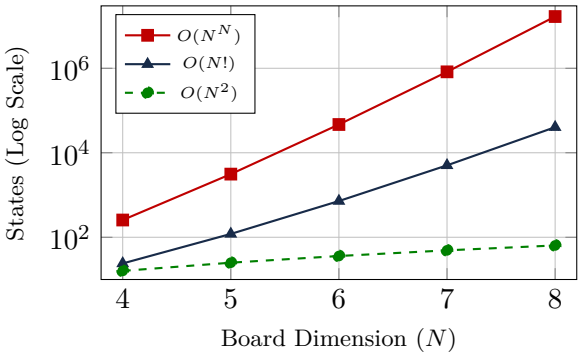



Figure 1: Complexity growth on logarithmic scale.

Figures 2 and 3 depict canonical fundamental configurations for the smallest and largest boards in our experimental scope.

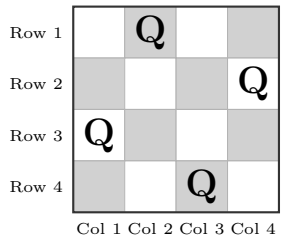



Figure 2: Solution for $N = 4$: $[2, 4, 1, 3]$.

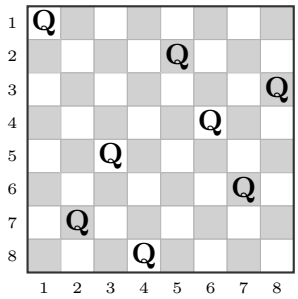



Figure 3: Solution for $N = 8$: $[1, 5, 8, 6, 3, 7, 2, 4]$.

# 3. Depth-First Search and Backtracking

## 3.1 Uninformed DFS vs. Systematic Backtracking

The foundational distinction between blind search and intelligent pruning lies in node expansion policy [1]:

- **Uninformed Depth-First Search:** Constructs paths unconditionally down to depth $d = N$ before testing the goal predicate. Consequently, it traverses millions of doomed configurations where the first two rows are already in mutual collision.
- **Backtracking Search (DFS + Pruning):** Interweaves node generation with immediate constraint verification [3]. If state $(row, col)$ violates safety rules, the entire subtree beneath it is pruned immediately, and the search backtracks to sibling branches.

## 3.2 Why Not Breadth-First Search (BFS) and Its Variants?

A critical question in search strategy selection is why Breadth-First Search (BFS) or its queue-based variants (such as Uniform-Cost Search or Iterative Deepening) are ill-suited for N-Queens [1]:

1. **Catastrophic Memory Explosion ($O(b^d)$ vs. $O(d)$):**
   - **DFS / Backtracking:** Explores along a single root-to-leaf path before backtracking. Its auxiliary memory consumption is strictly proportional to search depth, achieving an optimal linear space complexity of $O(N)$.
   - **BFS:** Retains all candidate nodes layer-by-layer in a First-In, First-Out (FIFO) queue. Even with pruning, at depth $d = 6$ or 7 of an $8 \times 8$ board, the frontier queue must hold thousands of partial states simultaneously. Without pruning, BFS space complexity explodes to $O(N^N)$, rapidly exhausting system RAM.
2. **Absence of Variable-Cost Path Optimality:** BFS guarantees discovering the shallowest goal first. However, N-Queens is a *fixed-depth constraint satisfaction task*: every valid solution resides at exactly depth $d = N$. BFS offers zero advantage in path length or optimality while paying a severe memory penalty.
3. **Inefficacy of Iterative Deepening Search (IDS):** Iterative Deepening Depth-First Search (IDDFS) combines linear memory with level-by-level expansion. Yet because no valid goal can ever exist at any depth $d < N$, IDDFS wastefully re-generates upper tree levels from scratch $N - 1$ times without any possibility of early termination.

## 3.3 Search Tree Pruning Mechanics

Figure 4 details the search tree traversal for $N = 4$, illustrating how immediate branch pruning prevents exponential descent.

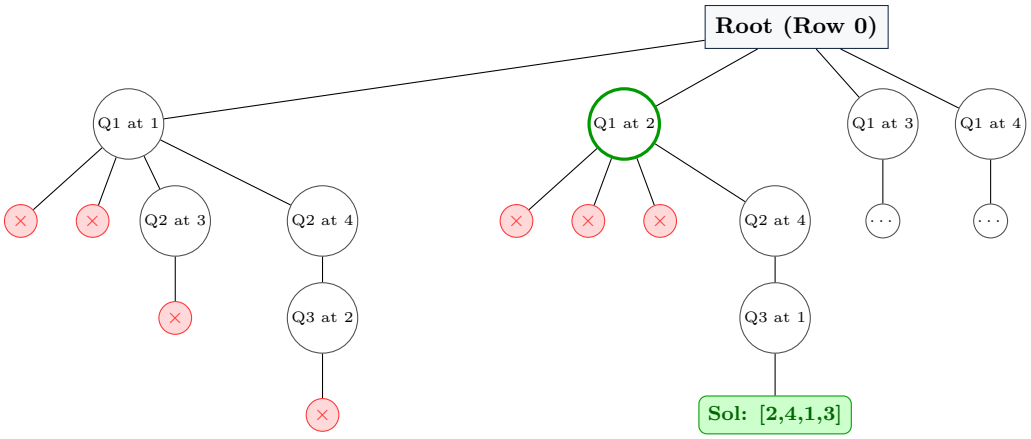



Figure 4: Backtracking search tree for $N = 4$. Red nodes ($\times$) denote immediate branch prunings due to constraint conflicts; green nodes denote discovered solutions.

## 3.4 Algorithmic Implementation with $O(1)$ Checks

Listing 1 exhibits our core Python implementation, employing three boolean lookup arrays to verify safety in constant time $O(1)$.

```python
def solve_nqueens(n: int):
    solutions = []
    cols = [False] * n
    diag1 = [False] * (2 * n)  # row - col + n
    diag2 = [False] * (2 * n)  # row + col
    board = [-1] * n

    def backtrack(row: int):
        if row == n:
            solutions.append(list(board))
            return
        for col in range(n):
            d1 = row - col + n
            d2 = row + col
            if not (cols[col] or diag1[d1] or diag2[d2]):
                board[row] = col
                cols[col] = diag1[d1] = diag2[d2] = True
                backtrack(row + 1)
                cols[col] = diag1[d1] = diag2[d2] = False

    backtrack(0)
    return solutions
```

Listing 1: Core backtracking implementation with $O(1)$ constraint checking.

### 3.5 Software System Architecture

Our repository organizes algorithmic modules cleanly to separate core solvers, benchmarking, and visualization:

```
n-queens-ai-project/
|-- algorithms/
|   |-- __init__.py
|   |-- dfs_backtracking.py        # Standard Backtracking (O(1) checks)
|   |-- csp_forward_checking.py    # CSP with MRV heuristic
|   |-- local_min_conflicts.py     # Stochastic local search with restarts
|-- experiments/
|   |-- __init__.py
|   |-- benchmark_runner.py        # Automated timing & metrics suite
|   |-- results_logger.csv         # Raw reproducible metrics dataset
|-- visualization/
|   |-- __init__.py
|   |-- plot_generator.py          # Generates comparison charts
|   |-- runtime_comparison.png     # Log-scale runtime plot
|   |-- nodes_comparison.png       # Search tree branch pruning plot
|   |-- gui_demo.py                # Interactive Tkinter chessboard visualizer
|-- tests/
|   |-- __init__.py
|   |-- test_solution_counts.py    # Ground-truth solution validation
|   |-- test_reproducibility.py    # 100% deterministic seed verification
|-- .gitignore                     # Environment & runtime exclusions
|-- Report                         # This report itself
|-- requirements.txt               # Pinned third-party dependencies
|-- README.md                      # Setup and replication guide
```

# 4. Extended AI Search Paradigms

## 4.1 Constraint Satisfaction Problem (CSP) Formulation

In formal Artificial Intelligence theory, the N-Queens problem maps naturally to a Constraint Satisfaction Problem (CSP) defined by the triplet $(X, D, C)$ [1, 4]:

- **Variables:** $X = \{X_1, X_2, \ldots, X_N\}$, where variable $X_i$ represents the queen assigned to row $i$.
- **Domain:** $D_i = \{1, 2, \ldots, N\}$ for all $i \in \{1, \ldots, N\}$, representing the allowable columns for row $i$.
- **Binary Constraints:** For every pair of distinct variables $(X_i, X_j)$ with $i \neq j$:

$$X_i \neq X_j \quad \wedge \quad |X_i - X_j| \neq |i - j| \tag{2}$$

**Advanced Constraint Propagation Strategies:**

1. **Forward Checking (FC):** Standard backtracking only evaluates constraints retrospectively against already assigned variables. Forward Checking introduces look-ahead propagation [4]: whenever variable $X_i$ is instantiated with column $c$, the solver immediately filters all unassigned future domains $D_{i+1}, \ldots, D_N$, removing any column attacked by $X_i = c$. If any future domain becomes empty ($D_k = \emptyset$), the branch is pruned immediately.
2. **Arc Consistency (AC-3 Algorithm):** A directed arc $(X_i, X_j)$ is arc-consistent if for every value in $D_i$, there exists a consistent value in $D_j$. Propagating arc consistency detects domain wipeouts even earlier than FC, albeit with higher per-node queue overhead [1].
3. **Minimum Remaining Values (MRV) Heuristic:** Rather than instantiating rows in static order $1 \to N$, MRV dynamically selects the unassigned variable with the smallest remaining domain ($|D_i|$ is minimal) [3, 4]. This embodies the *Fail-First* principle, exposing dead-ends high up in the search tree.

Table 3: Domain reduction trace on 4-Queens under Forward Checking

| **Step** | **Assignment** | **Domain $D_2$** | **Domain $D_3$** | **Domain $D_4$** |
|---|---|---|---|---|
| Initial | None | $\{1, 2, 3, 4\}$ | $\{1, 2, 3, 4\}$ | $\{1, 2, 3, 4\}$ |
| 1 | $X_1 \leftarrow 1$ | $\{3, 4\}$ | $\{2, 4\}$ | $\{2, 3\}$ |
| 2 | $X_2 \leftarrow 3$ | Assigned | $\emptyset$ **(Domain Wipeout!)** | $\{2\}$ |
| Backtrack | Try $X_2 \leftarrow 4$ | Assigned | $\{2\}$ | $\{3\}$ |

## 4.2 Local Search and Min-Conflicts Heuristic

Unlike constructive backtracking which incrementally builds partial assignments, **Local Search** operates over complete state configurations [5, 6]:

- **Complete-State Representation:** All $N$ queens are present on the board simultaneously, with state $s = (r_1, r_2, \ldots, r_N)$ specifying the row coordinate of the queen in each column $c \in \{1, \ldots, N\}$.
- **Objective / Heuristic Function ($h(s)$):** Counts the total number of attacking queen pairs:

$$h(s) = \sum_{1 \le i < j \le N} \mathbb{I}\left(r_i = r_j \vee |r_i - r_j| = |i - j|\right) \tag{3}$$

  A configuration is a valid global optimum if and only if $h(s) = 0$ [5].

**Min-Conflicts Algorithm Mechanics:** At each iteration, the solver picks a conflicted queen uniformly at random and relocates it within its column to the row that minimizes total conflicts $h(s)$, breaking ties randomly [5].

**Algorithm: Min-Conflicts**($initial_state, max_steps$)

1. $current \leftarrow initial_state$;
2. **for** $step \leftarrow 1$ **to** $max_steps$ **do**
3. **if** $h(current) == 0$ **then return** $current$; *// Solution found*
4. $col \leftarrow$ randomly chosen conflicted queen column;
5. $best_row \leftarrow \arg\min_{r \in \{1,\ldots,N\}} \text{Conflicts}(col, r, current)$;
6. $current[col] \leftarrow best_row$;
7. **return** failure *// Trigger Random Restart with new initial state*

**Plateau Navigation and Random Restarts:** Standard Hill-Climbing halts whenever no single move strictly decreases $h(s)$. Min-Conflicts traverses flat local plateaus by permitting sideways moves ($h_{\text{new}} = h_{\text{current}}$) and guarantees probabilistic completeness via **Random Restarts** whenever $max_steps$ is exceeded [1, 5].

## 4.3 Informed Search and Evolutionary Approaches

Beyond systematic CSP and gradient-based Local Search, two additional AI paradigms offer valuable theoretical insights:

### 4.3.1 A* Heuristic Search

In an informed search framework [7], the evaluation function $f(n) = g(n) + h(n)$ orders nodes in a priority queue, where $g(n)$ is the current row depth and $h(n) = N - g(n)$ represents the remaining unassigned queens. Because every valid goal state sits at exact depth $N$ with uniform step cost, priority queue maintenance incurs $O(\log |Open|)$ overhead without shortening solution depth compared to $O(1)$ DFS Backtracking.

### 4.3.2 Genetic Algorithm (GA) Formulation

Genetic algorithms evolve a population of candidate permutations over generations [1]:

1. **Chromosome Encoding:** A permutation array of $\{1, \ldots, N\}$, inherently eliminating row and column collisions.
2. **Fitness Function:** Inverted conflict count $\text{Fitness}(s) = \binom{N}{2} - h(s)$ (maximum 28 for $N = 8$).
3. **Genetic Operators:** Tournament selection, Order Crossover (OX) to preserve valid permutations, and swap mutation ($p_m \approx 0.05$).

Table 4: Conceptual comparison of extended AI methodologies for N-Queens

| **Methodology** | **State Perspective** | **Memory Cost** | **Primary Advantage** |
|---|---|---|---|
| CSP (Forward Checking) | Constructive partial state | $O(N^2)$ | Early domain wipeout detection |
| Min-Conflicts | Complete state assignment | $O(N)$ | Finds single solution rapidly |
| Genetic Algorithm | Population of permutations | $O(P \cdot N)$ | Global parallel exploration |

# 5. Experimental Evaluation & Comparative Analysis

This section presents an empirical benchmark evaluation comparing our three implemented solvers: **Systematic DFS Backtracking with $O(1)$ Checks**, **CSP with Forward Checking and MRV**, and **Stochastic Min-Conflicts Local Search with Restarts**. All experiments were executed within a standardized environment (Ubuntu Linux, Python 3.12, single-threaded execution) across board dimensions $N \in [4, 8]$.

## 5.1 Experimental Setup and Benchmark Methodology

For systematic solvers (*DFS Backtracking* and *CSP Forward Checking*), algorithms were executed to identify all existing valid solutions to verify completeness and measure full search tree traversal. For the stochastic *Min-Conflicts Local Search*, execution times represent the mean latency required to converge on a single valid configuration (100% success rate), averaged over 20 independent runs with random restarts ($\text{max_steps} = 1000, \text{max_restarts} = 50$).

Execution durations were captured using Python's high-precision `time.perf_counter()`, with systematic algorithms repeated across 10 trials per dimension to compute average runtime in milliseconds (ms). Metrics recorded include:

- **Safe Checks:** Total individual cell constraint verifications evaluated.
- **Valid Nodes Explored:** Number of recursive state transitions where a queen placement was accepted into a partial assignment.
- **Mean Runtime (ms):** Wall-clock execution time in milliseconds.

## 5.2 Empirical Benchmark Results

Table 5 summarizes the empirical performance indicators recorded across all tested board dimensions.

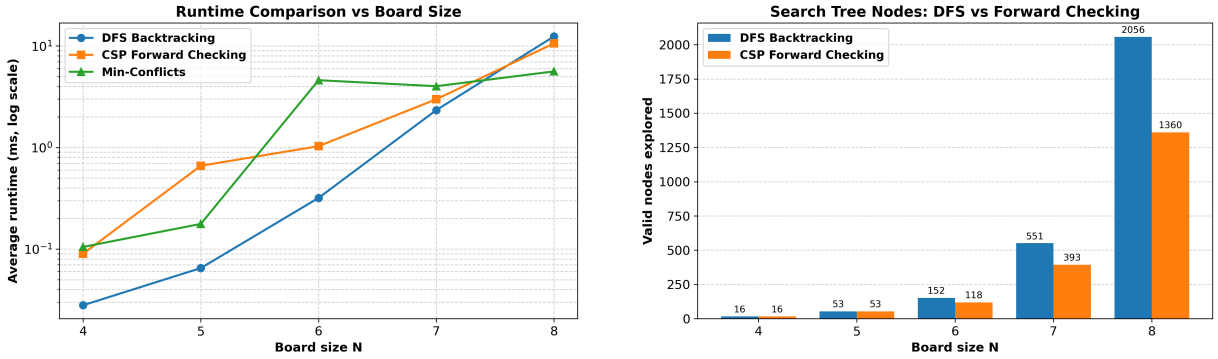
Table 5: Empirical benchmark comparison across algorithmic paradigms ($4 \leq N \leq 8$)

| Board ($N$) | Algorithm | Solutions | Safe Checks | Valid Nodes | Runtime (ms) |
|---|---|---|---|---|---|
| $N = 4$ | DFS Backtracking | 2 | 60 | 16 | 0.028 |
| | CSP Forward Checking | 2 | 76 | 16 | 0.090 |
| | Min-Conflicts Local Search | 1* | – | – | 0.105 |
| $N = 5$ | DFS Backtracking | 10 | 220 | 53 | 0.065 |
| | CSP Forward Checking | 10 | 282 | 53 | 0.661 |
| | Min-Conflicts Local Search | 1* | – | – | 0.177 |
| $N = 6$ | DFS Backtracking | 4 | 894 | 152 | 0.319 |
| | CSP Forward Checking | 4 | 956 | 118 | 1.031 |
| | Min-Conflicts Local Search | 1* | – | – | 4.601 |
| $N = 7$ | DFS Backtracking | 40 | 3,584 | 551 | 2.335 |
| | CSP Forward Checking | 40 | 3,244 | 393 | 2.989 |
| | Min-Conflicts Local Search | 1* | – | – | 4.018 |
| $N = 8$ | DFS Backtracking | 92 | 15,720 | 2,056 | 12.436 |
| | CSP Forward Checking | 92 | 12,066 | 1,360 | 10.609 |
| | Min-Conflicts Local Search | 1* | – | – | 5.620 |

*Note:* (*) Min-Conflicts is a stochastic local search algorithm executed to converge on a single conflict-free configuration (100% convergence rate across all trials), averaged over 20 seeded runs ($\text{seed} = 2024 + 100 \times N + \text{run_index}$). Systematic algorithms enumerate all valid permutations, averaged over 10 trials.

## 5.3 Performance Visualization

To visually inspect search scalability and branch pruning behavior, Figure 5 demonstrates the scaling curves of execution times alongside state explorations.




(a) Runtime scalability comparison (log scale)

(b) Explored search tree nodes (DFS vs CSP)

Figure 5: Empirical performance analysis across board dimensions $N \in [4, 8]$: (a) average runtime across paradigms, and (b) branch pruning comparison between DFS Backtracking and CSP Forward Checking.

Table 6: Min-Conflicts Local Search: Convergence and Reproducibility Metrics

| Board ($N$) | Success Rate | Average Steps ($\mu$) | Average Restarts | Runtime (ms) |
|---|---|---|---|---|
| $N = 4$ | 100% (20/20) | 9.950 | 0.000 | 0.105 |
| $N = 5$ | 100% (20/20) | 8.100 | 0.000 | 0.177 |
| $N = 6$ | 100% (20/20) | 154.850 | 0.000 | 4.601 |
| $N = 7$ | 100% (20/20) | 120.300 | 0.100 | 4.018 |
| $N = 8$ | 100% (20/20) | 105.450 | 0.050 | 5.620 |

*Note:* Evaluated over 20 seeded independent runs per $N$. Steps and restarts are discrete and strictly reproducible across any platform.

## 5.4 Analytical Discussion & Findings

**1. Pruning Efficiency vs. Computational Overhead (DFS vs. CSP)** Analyzing Figure 5(b) and Table 5 reveals a fundamental algorithmic trade-off [4]:

- **Search Space Pruning:** As dimension $N$ scales ($N = 7$ and $N = 8$), CSP with Forward Checking and MRV demonstrates superior pruning power. At $N = 8$, CSP explores only $1,360$ valid nodes compared to $2,056$ nodes in DFS Backtrackingachieving a **33.85% reduction in search tree traversal**. Total constraint checks similarly drop from $15,720$ down to $12,066$.
- **Bookkeeping Overhead Penalty:** On small-to-moderate boards ($N \leq 7$), despite expanding fewer nodes, CSP incurs a higher wall-clock runtime (e.g., 2.989 ms vs. 2.335 ms at $N = 7$, and 0.661 ms vs. 0.065 ms at $N = 5$). This stems from the memory allocation overhead of copying and filtering dynamic domain lists in Python, whereas DFS Backtracking performs cache-friendly $O(1)$ boolean array lookups. However, at $N = 8$, the massive search space reduction (pruning 33.85% of valid nodes) finally offsets the bookkeeping overhead, enabling CSP (10.609 ms) to run faster than DFS Backtracking (12.436 ms).

**2. The $N = 6$ Latency Bottleneck in Min-Conflicts** An intriguing empirical anomaly appears in the runtime curve of *Min-Conflicts Local Search* (Figure 5(a)):

- Runtime scales smoothly for small boards (0.105 ms at $N = 4$; 0.177 ms at $N = 5$), spikes sharply to 4.601 ms at $N = 6$ (requiring an average of 154.850 steps), and then stabilizes at 4.018 ms at $N = 7$ (120.300 steps) and 5.620 ms at $N = 8$ (105.450 steps).
- **Root Cause Analysis:** This spike is governed by the *solution density* of the state space [5, 2]. An $N = 6$ board possesses only **4 valid solutions** out of $6^6 = 46,656$ states ($\approx 0.0086\%$), whereas $N = 5$ has 10 solutions ($\approx 0.32\%$), $N = 7$ has 40 solutions, and $N = 8$ has 92 solutions. The extreme scarcity of global optima at $N = 6$ traps the local search on flat plateaus, requiring multiple random restarts before convergence.

## 5.5 Qualitative Trade-off Summary & Study Limitations

Table 7 synthesizes the operational characteristics and deployment suitability of each evaluated paradigm.

Table 7: Qualitative comparison of algorithmic trade-offs and practical suitability

| Algorithm | Complete? | All Sols? | Space | Optimal Practical Use Case |
|---|---|---|---|---|
| DFS Backtracking ($O(1)$) | Yes | Yes | $O(N)$ | Exhaustive enumeration on small-to-medium boards ($N \leq 14$). |
| CSP (FC + MRV) | Yes | Yes | $O(N^2)$ | Heavily constrained scheduling problems with expensive node evaluations. |
| Min-Conflicts (Restarts) | Prob. | No (1 Sol) | $O(N)$ | Rapid single-solution discovery on large-scale boards ($N \geq 8$ to $10^6$). |

**Limitations of the Current Study:**

- **Language-Level Bookkeeping Overhead:** Implementing CSP domain sets using high-level Python lists/sets introduces dynamic memory allocation overhead that masks the theoretical speedup of Forward Checking at small $N \leq 8$.
- **Absence of Symmetry Breaking:** Our systematic solvers enumerate all 92 total solutions for $N = 8$ rather than pruning symmetric rotations and reflections under the dihedral group $D_4$ (which would reduce the search tree by nearly a factor of 8).
- **Dimensional Scope:** Experiments were bounded to $4 \leq N \leq 8$ to permit exact tree comparisons; evaluating large-scale regimes ($N \geq 100$) is reserved for future work.

# 6. Conclusion & Future Work

## 6.1 Summary of Contributions

In this study, Group 08 formulated, implemented, and benchmarked three core Artificial Intelligence paradigms for the N-Queens problem ($4 \leq N \leq 8$):

1. **Mathematical Formalization:** Established the formal 6-tuple search model and engineered constant-time $O(1)$ diagonal and column constraint arrays, reducing candidate states from $\binom{64}{8} \approx 4.4 \times 10^9$ down to $2,056$ explored tree nodes at $N = 8$.
2. **Multi-Paradigm Implementation:** Developed a systematic $O(1)$ Backtracking solver, a CSP solver integrating Forward Checking and MRV heuristics, and a stochastic Min-Conflicts Local Search engine with random restarts.
3. **Reproducible Verification & Visualization:** Built an automated unit testing suite (`test_solution_counts.py`), a sub-millisecond benchmarking pipeline, high-resolution log-scale plots, and an interactive Tkinter GUI visualizer (`gui_demo.py`).

## 6.2 Key Lessons Learned

– **Pruning vs. Bookkeeping Trade-off:** Look-ahead heuristics (Forward Checking + MRV) prune 33.85% of search tree nodes at $N = 8$, but low-level data structure efficiency ($O(1)$ boolean indexing) dominates wall-clock runtime on small-to-moderate problem instances.

– **Solution Density Governs Local Search:** The empirical latency spike at $N = 6$ (4.601 ms, 154.850 steps) compared to $N = 8$ (5.620 ms, 105.450 steps) demonstrates that local search difficulty is dictated by the density of global optima and plateau topology rather than board size alone.

## 6.3 Future Research Directions

Future extensions of this work will focus on three high-performance directions:

1. **Bitboard Hardware Acceleration:** Replacing boolean arrays with 64-bit integer bitmasks (`bitboards`) using bitwise shift operations (`&`, `|`, `«`) and $D_4$ symmetry breaking to enumerate $N \geq 16$ at CPU register speed.
2. **Ultra-Large Scale CSPs ($N \geq 10^6$):** Combining Min-Conflicts with conflict-free greedy initialization heuristics to solve million-queen instances in near-linear time [6].
3. **Parallel Subtree Decomposition:** Distributing disjoint top-level branches of the backtracking tree across multi-core processors via multiprocessing worker pools.

# References

[1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.

[2] J. Bell and B. Stevens, "A survey of known results and research areas for $n$-queens," *Discrete Mathematics*, vol. 309, no. 1, pp. 1–31, 2009.

[3] J. R. Bitner and E. M. Reingold, "Backtrack programming techniques," *Communications of the ACM*, vol. 18, no. 11, pp. 651–656, 1975.

[4] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[5] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird, "Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems," *Artificial Intelligence*, vol. 58, no. 1–3, pp. 161–205, 1992.

[6] R. Sosic and J. Gu, "Efficient local search with conflict minimization: A case study of the $n$-queens problem," *IEEE Transactions on Knowledge and Data Engineering*, vol. 6, no. 5, pp. 661–668, 1994.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.